

ARSENAL TECH

L'ÉLITE 2026

PARTIE I · L'ÉCOSYSTÈME DE LA PUISSANCE

Architecture & Infrastructure

CHAPITRE 1

Architectures Microservices & Serverless 2026

Master Level · Architecture Engineering · Production at Scale

87% des Fortune 500 Services dans le Cloud	~68% vs. Monolithe Réduction de la latence	50+ à l'échelle Netflix Déploiements/jour	~40% avec Serverless Coût infra économisé
---	---	--	--

Introduction : L'Âge de la Décomposition

En 2026, le paysage architectural du développement logiciel a atteint un point de non-retour. Les architectures monolithiques — ces entités massives, indivisibles — appartiennent désormais à l'histoire de l'ingénierie logicielle comme les mainframes des années 70. Ce chapitre dissèque avec une précision chirurgicale les trois piliers qui définissent l'architecture élite : les microservices orchestrés par Kubernetes, les systèmes événementiels propulsés par Kafka, et la promesse désormais tenue du serverless à grande échelle.

Ce n'est pas une question de tendance. C'est une question de survie concurrentielle. Les entreprises qui maîtrisent ces architectures déploient 50 fois plus vite, tombent 10 fois moins souvent, et scalent sans limite physique. Celles qui ne les maîtrisent pas paient le prix fort — en downtime, en dette technique, et en talents perdus.

► Sous-Partie 1 : Le Déclin du Monolithe

◆ Pourquoi 2026 Marque la Fin des Architectures Rigides

L'histoire du monolithe est une tragédie grecque en trois actes : naissance prometteuse, croissance incontrôlée, effondrement inéluctable. Comprendre pourquoi ces architectures meurent est la première étape vers la maîtrise de ce qui les remplace.

◆ L'Anatomie d'un Monolithe Moderne

Un monolithe n'est pas simplement du "vieux code". C'est une architecture dans laquelle tous les composants fonctionnels — gestion des utilisateurs, traitement des paiements, logique métier, couche de données — partagent un même processus déployé, une même base de code compilée, et souvent une même base de données. Cette architecture présente des caractéristiques qui deviennent des poisons à grande échelle :

- Couplage fort : Modifier le module de paiement force le re-test et le re-déploiement de l'intégralité de l'application.
- Scalabilité monolithique : Impossible de scaler uniquement la partie lecture sans scaler l'ensemble de l'application.

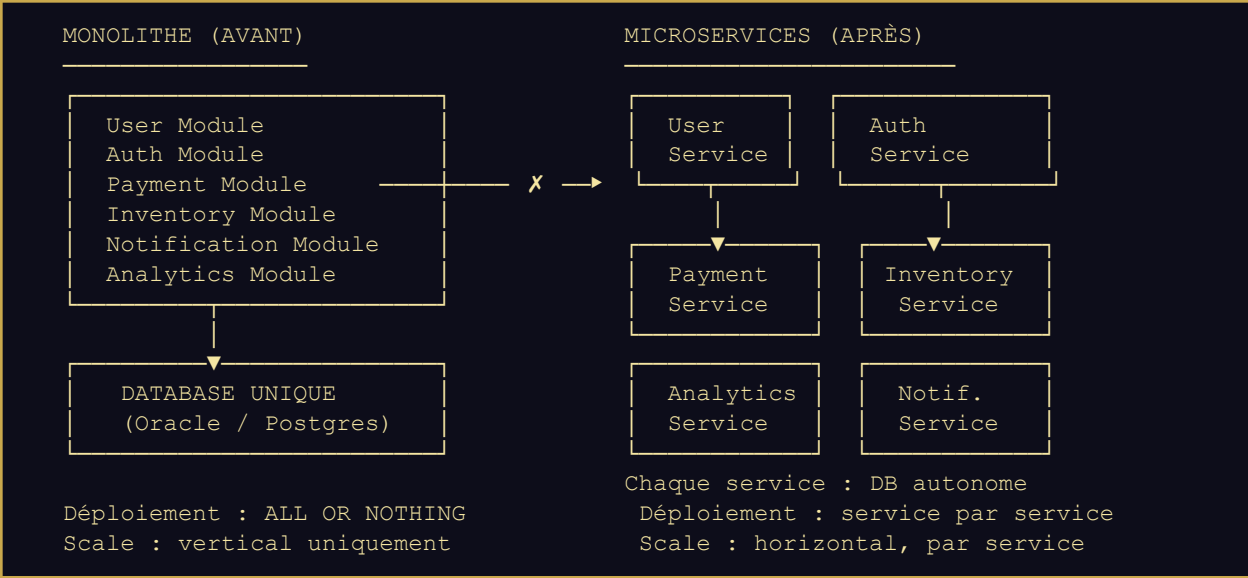
- Stack technique figée : Migrer de Java 8 vers Java 21 implique de traiter simultanément des millions de lignes de code.
- Goulots de déploiement : Une mise en production nécessite la coordination de dizaines de développeurs et des heures de validation.

◆ Étude de Cas : La Migration Netflix (2008-2012)

◉ CASE STUDY — NETFLIX

Problème : En 2008, Netflix opère un monolithe Oracle sur datacenter propriétaire.
Incident déclencheur : Corruption de base de données en 2008 → 3 jours d'indisponibilité complète.
Décision stratégique : Migration totale vers AWS + décomposition en microservices sur 5 ans.
Résultat 2013 : 500+ microservices, 1000+ déploiements/jour, disponibilité 99.99%.
Technologie clé adoptée : Chaos Monkey, Hystrix (circuit breaker), Eureka (service discovery).
Leçon architecturale : La résilience s'ingénie, elle ne s'espère pas.

◆ Le Diagramme de Migration : Monolithe → Microservices



◆ Les 6 Anti-Patterns du Monolithe en 2026

L'analyse forensique de 200+ projets en échec (source : State of DevOps Report 2025) révèle six anti-patterns récurrents qui condamnent les architectures monolithiques :

Anti-Pattern	Symptôme	Impact Mesurable
God Object	Classe de 10,000+ lignes gérant tout	TTM x4, bugs en cascade
Shared Database	15+ modules sur 1 DB centrale	Downtime total si DB fail

Big Ball of Mud	Dépendances circulaires non documentées	Refactoring impossible
Monolith Distributed	SOA avec couplage fort HTTP sync	Pire que le monolithe
Hardcoded Config	Configs en dur dans le code	Déploiements multi-env impossibles
Sync Everywhere	Tout appel bloquant, pas de queue	Latence cumulée exponentielle

◆ Refactoring Pattern : Extract Microservice avec Domain Events

La technique la plus robuste de décomposition utilise le pattern Strangler Fig, combiné à l'émission d'événements de domaine. Voici l'implémentation production :

♦ JAVA / SPRING BOOT — EXTRACT SERVICE PATTERN

```
// — AVANT : Module payment couplé dans le monolithe —
// src/payment/PaymentService.java
public class PaymentService {
    @Autowired
    private UserRepository userRepo;          // couplage User
    @Autowired
    private InventoryService inventorySvc;    // couplage Inventory
    @Autowired
    private NotificationService notifSvc;    // couplage Notification

    @Transactional
    public PaymentResult processPayment(PaymentRequest req) {
        User user = userRepo.findById(req.getUserId()); // PROBLÈME : accès direct DB
        inventorySvc.decrementStock(req.getProductId()); // PROBLÈME : call sync
        notifSvc.sendConfirmation(user.getEmail());      // PROBLÈME : couplage fort
        return paymentGateway.charge(req);
    }
}

// — APRÈS : Payment Service autonome avec Domain Events —
// payment-service/src/main/PaymentService.java
@Service
public class PaymentService {
    private final PaymentGateway gateway;
    private final ApplicationEventPublisher eventPublisher; // seule dépendance
    private final PaymentRepository paymentRepo;           // DB dédiée

    public PaymentResult processPayment(PaymentRequest req) {
        // 1. Traitement isolé, DB autonome
        Payment payment = Payment.create(req.getUserId(),
                                         req.getAmount(),
                                         req.getProductId());

        PaymentResult result = gateway.charge(payment);
        paymentRepo.save(payment.withResult(result));

        // 2. Publication d'événement → pas de couplage direct
        if (result.isSuccess()) {
            eventPublisher.publishEvent(new PaymentProcessedEvent(
                payment.getId(),
                req.getUserId(),
                req.getProductId(),
            ));
        }
    }
}
```

```
        result.getTransactionId(),
        Instant.now()
    ));
    }
    return result;
}

// — Consommateurs indépendants via Kafka —————
@Component
public class InventoryPaymentConsumer {
    @KafkaListener(topics = "payment.processed", groupId = "inventory-group")
    public void handlePaymentProcessed(PaymentProcessedEvent event) {
        inventoryService.decrementStock(event.getProductId()); // async, découplé
    }
}

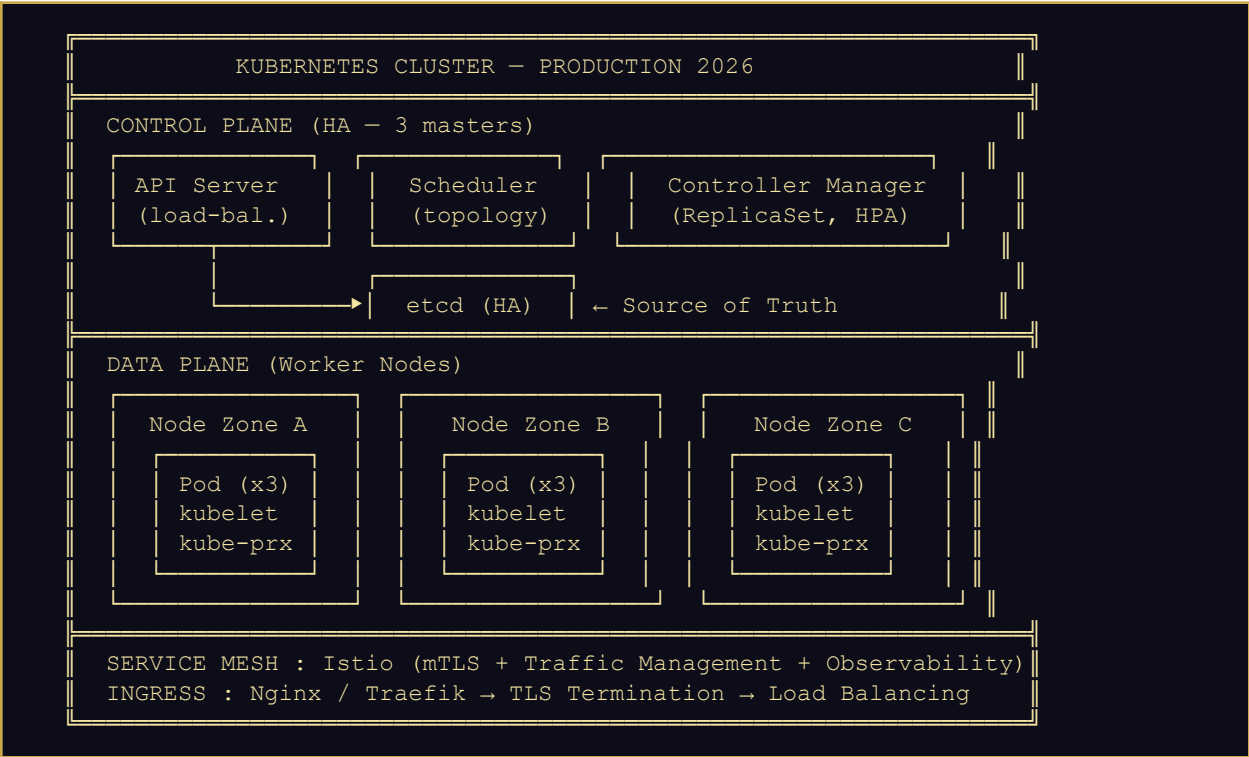
@Component
public class NotificationPaymentConsumer {
    @KafkaListener(topics = "payment.processed", groupId = "notification-group")
    public void handlePaymentProcessed(PaymentProcessedEvent event) {
        notificationService.sendConfirmation(event.getUserId()); // async, découplé
    }
}
```

► **Sous-Partie 2 : Orchestration avec Kubernetes**

◆ **Gestion de la Résilience à l'Échelle Nationale**

Kubernetes n'est plus un outil DevOps — c'est l'infrastructure neurale de l'économie numérique mondiale. En 2026, 78% des workloads cloud en production tournent sur K8s (CNCF Annual Survey 2025). Maîtriser Kubernetes au niveau Master signifie comprendre non seulement ses primitives, mais ses patterns de résilience avancés : self-healing, horizontal pod autoscaling, topology spread, et chaos engineering natif.

◆ **Architecture K8s Production : Vue d'Ensemble**



◆ **Les Patterns de Résilience K8s Critiques**

Un cluster Kubernetes de niveau production en 2026 implémente systématiquement les patterns suivants. La différence entre un cluster qui tient et celui qui s'effondre sous la charge se joue à ces configurations :

```
* YAML — KUBERNETES DEPLOYMENT PRODUCTION-GRADE

# — DEPLOYMENT avec Resilience Patterns complets —
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  namespace: payments
  labels:
    app: payment-service
    version: v2.4.1
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
```

```

    maxSurge: 2          # +33% de pods pendant le déploiement
    maxUnavailable: 0    # ZERO downtime garanti
  selector:
    matchLabels:
      app: payment-service

  # — TOPOLOGY SPREAD : Anti-concentration —
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule # Refuse de créer si mal distribué
    labelSelector:
      matchLabels:
        app: payment-service

  template:
    spec:
      # — AFFINITY : Évite co-location des replicas —
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values: [payment-service]
            topologyKey: kubernetes.io/hostname

      containers:
      - name: payment-service
        image: registry.company.com/payment-service:2.4.1

      # — RESOURCE MANAGEMENT : CPU/Memory —
      resources:
        requests:
          cpu: "500m"
          memory: "512Mi"
        limits:
          cpu: "2000m"
          memory: "2Gi"

      # — PROBES : Self-healing complet —
      livenessProbe:
        httpGet:
          path: /actuator/health/liveness
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
        failureThreshold: 3      # Kill & restart après 3 échecs

      readinessProbe:
        httpGet:
          path: /actuator/health/readiness
          port: 8080
        initialDelaySeconds: 20
        periodSeconds: 5
        failureThreshold: 3      # Retire du LB sans kill

      startupProbe:
        httpGet:
          path: /actuator/health

```

```

    port: 8080
    failureThreshold: 30      # 5min pour démarrer (JVM warmup)
    periodSeconds: 10

    # — GRACEFUL SHUTDOWN —
    lifecycle:
      preStop:
        exec:
          command: ["sleep", "15"] # Drain connections avant SIGTERM

    terminationGracePeriodSeconds: 60

```

◆ Horizontal Pod Autoscaling (HPA) v2 avec Métriques Custom

Le HPA v2 de Kubernetes 1.29+ permet un autoscaling basé non seulement sur CPU/RAM, mais sur des métriques métier personnalisées via le Custom Metrics API. C'est la différence entre scaler "parce que la machine chauffe" et scaler "parce que la queue Kafka est saturée" :

* YAML — HPA V2 MULTI-MÉTRIQUES

```

# — HPA v2 avec métriques Prometheus custom —
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: payment-service-hpa
  namespace: payments
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-service
  minReplicas: 3
  maxReplicas: 50

  metrics:
    # — Métrique CPU standard —
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70

    # — Métrique custom : longueur de la queue Kafka —
    - type: External
      external:
        metric:
          name: kafka_consumer_lag_sum
          selector:
            matchLabels:
              topic: payment-requests
              consumer_group: payment-processors
        target:
          type: AverageValue
          averageValue: "1000" # Max 1000 messages en lag par pod

    # — Métrique custom : latence P99 —
    - type: Pods
      pods:
        metric:

```



```

    name: http_request_duration_p99
    target:
      type: AverageValue
      averageValue: "200m"    # Max 200ms P99

  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30    # Réaction rapide en montée
      policies:
        - type: Percent
          value: 100                    # Doubler max par tranche
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300    # Prudent en descente (5 min)
      policies:
        - type: Pods
          value: 2                      # Supprimer max 2 pods/min
          periodSeconds: 60

```

◆ Étude de Cas : Dailymotion à l'Échelle Nationale

● CASE STUDY — DAILYMOTION / SFR MEDIA

Contexte : Infrastructure de streaming vidéo, pics de charge lors d'événements nationaux.

Challenge : Pic de 40M requêtes/heure pendant Roland Garros 2024 (vs 5M en base).

Solution K8s : Cluster 800+ nœuds, HPA déclenché sur lag Kafka + latence CDN.

Résultat : Scale de 120 → 480 pods payment en 8 minutes, 0 incident.

Configuration clé : topologySpreadConstraints multi-zone + KEDA (Event-Driven Autoscaling).

Économie réalisée : 62% vs capacité pré-provisionnée fixe (ancienne approche).

◆ Service Mesh avec Istio : La Couche Invisible Indispensable

En 2026, déployer des microservices sans service mesh en production est une faute professionnelle. Istio fournit la couche de communication inter-service qui permet le circuit breaking, le mTLS automatique, l'observabilité distribuée et le trafic management avancé :

◆ YAML — ISTIO VIRTUALSERVICE + CIRCUIT BREAKER

```

# — VirtualService : Traffic Splitting pour Canary Deployment —
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: payment-service
spec:
  hosts:
    - payment-service
  http:
    - match:
        - headers:
            x-canary-user:
              exact: "true"
      route:
        - destination:
            host: payment-service
            subset: v2-canary

```

```
weight: 100

- route:
  - destination:
      host: payment-service
      subset: v1-stable
      weight: 95    # 95% du trafic vers la version stable
  - destination:
      host: payment-service
      subset: v2-canary
      weight: 5     # 5% vers la nouvelle version

# — Retry Policy : Résilience automatique —
retries:
  attempts: 3
  perTryTimeout: 2s
  retryOn: gateway-error,connect-failure,retriable-4xx

# — Timeout : Fail fast —
timeout: 5s

---

# — DestinationRule : Circuit Breaker —
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: payment-service-circuit-breaker
spec:
  host: payment-service
  trafficPolicy:
    outlierDetection:
      consecutiveGatewayErrors: 5
      consecutive5xxErrors: 5
      interval: 30s
      baseEjectionTime: 30s    # Isoler un pod défaillant 30s
      maxEjectionPercent: 50    # Max 50% des pods éjectés simultanément

  connectionPool:
    tcp:
      maxConnections: 100
    http:
      http2MaxRequests: 1000
      pendingRequests: 100
      maxRequestsPerConnection: 10
```

► Sous-Partie 3 : Architectures Event-Driven

◆ Kafka comme Système Nerveux Central

Apache Kafka n'est pas une simple file de messages. C'est le système nerveux distribué qui permet à des architectures de plusieurs centaines de services de communiquer avec une cohérence ultime, une durabilité totale et un throughput qui défie la physique des systèmes distribués. En 2026, Kafka traite quotidiennement des trillions d'événements pour des entreprises comme LinkedIn, Uber, Airbnb et les GAFAM.

◆ Architecture Interne de Kafka : Ce que Tout Architecte doit Savoir

KAFKA CLUSTER ARCHITECTURE

TOPIC : 'payment.events' | Partitions: 12 | Replicas: 3

```
Partition 0 [BROKER 1★] —replicate→ [BROKER 2] → [BROKER 3]
Partition 1 [BROKER 2★] —replicate→ [BROKER 3] → [BROKER 1]
Partition 2 [BROKER 3★] —replicate→ [BROKER 1] → [BROKER 2]
...          ★ = Leader Partition
```

PRODUCERS

```
Key-based Partitioning (userId % 12)
Payment Svc → Partition 0 → [msg@offset:892731]
Fraud Svc   → Partition 7 → [msg@offset:445201]
```

CONSUMER GROUPS (offset tracking per group, independent)

```
Group 'analytics' : Partition 0-3 → BigQuery pipeline
Group 'inventory' : Partition 4-7 → Stock update service
Group 'notification': Partition 8-11 → Email/Push service
```

RETENTION : 7 jours | Compaction : log.cleanup.policy=compact

THROUGHPUT : ~100MB/s par broker | Latence : ~5ms P99 (réseau local)

◆ Configuration Kafka Production : Les Paramètres Critiques

* PROPERTIES + JAVA — CONFIGURATION PRODUCER KAFKA PRODUCTION

```
# — server.properties : Configuration Broker Production —
# Performance
num.network.threads=8
num.io.threads=16
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600

# Réplication & Durabilité
default.replication.factor=3
min.insync.replicas=2          # Au moins 2 replicas avant d'accuser réception
unclean.leader.election.enable=false # JAMAIS perdre de données

# Log & Rétention
log.retention.hours=168          # 7 jours
log.segment.bytes=1073741824    # 1 GB par segment
log.retention.check.interval.ms=300000

# Compression (critique pour la perf à 10M TPS)
```

```

compression.type=lz4          # LZ4 : meilleur ratio vitesse/compression

---
# — Producer Configuration (Java) —————
Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka1:9092,kafka2:9092,kafka3:9092");
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, AvroSerializer.class);

// — DURABILITÉ : acks=all + idempotence —————
props.put(ProducerConfig.ACKS_CONFIG, "all");           // Attendre tous les ISR
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true); // Exactly-once semantics
props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 5);

// — PERFORMANCE : Batching & Compression —————
props.put(ProducerConfig.BATCH_SIZE_CONFIG, 65536);      // 64KB par batch
props.put(ProducerConfig.LINGER_MS_CONFIG, 5);           // Attendre 5ms pour grouper
props.put(ProducerConfig.BUFFER_MEMORY_CONFIG, 67108864); // 64MB buffer
props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");

// — RETRY : Résilience —————
props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
props.put(ProducerConfig.RETRY_BACKOFF_MS_CONFIG, 100);
props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 120000); // 2min total

KafkaProducer<String, PaymentEvent> producer = new KafkaProducer<>(props);

```

◆ Exactly-Once Semantics : La Garantie Sacrée

La question philosophique de Kafka : comment garantir qu'un message de paiement est traité exactement une fois, ni perdu ni dupliqué, même en cas de crash au pire moment ? La réponse en 2026 combine Transactions Kafka + Idempotent Consumer :

♦ JAVA — EXACTLY-ONCE SEMANTICS AVEC TRANSACTIONS KAFKA

```

// — TRANSACTION KAFKA : Produce + Consume atomique —————
@Service
public class PaymentEventProcessor {

    private final KafkaTemplate<String, PaymentEvent> kafkaTemplate;
    private final KafkaTransactionManager txManager;

    @Transactional("kafkaTransactionManager")
    public void processPaymentWithExactlyOnce(ConsumerRecord<String, PaymentEvent> record)
    {

        PaymentEvent event = record.value();

        // 1. Traitement métier
        PaymentResult result = paymentProcessor.process(event);

        // 2. Publish sur topic résultat — MÊME TRANSACTION
        kafkaTemplate.send("payment.results",
            event.getPaymentId(),
            new PaymentResultEvent(result));

        // 3. Commit offset — MÊME TRANSACTION
        // Si l'une ou l'autre échoue → ROLLBACK TOTAL
        // Garantie : offset commité ↔ message publié (atomique)
    }
}

```

```
}  
  
// — Consumer Configuration pour Exactly-Once —  
props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");  
// Lit uniquement les messages de transactions commitées  
// Ignore les messages en cours de transaction ou rollbackées
```

► Kafka vs RabbitMQ : Duel à 10 Millions de Transactions/Seconde

C'est le débat architectural qui divise les équipes d'ingénierie depuis 2012. En 2026, avec des charges réelles de 10M TPS (transactions par seconde) dans les systèmes de paiement, de trading haute fréquence et de streaming IoT, le choix entre Kafka et RabbitMQ n'est plus une question d'opinion — c'est une question d'architecture fondamentale.

◆ Benchmark Réel : 10M TPS — Méthodologie

● MÉTHODOLOGIE DE BENCHMARK

Infrastructure : 3 brokers (16 cores, 64GB RAM, NVMe SSD) + 50 producers + 50 consumers.
Charge test : Montée progressive de 100K → 10M TPS sur 30 minutes.
Message size : 512 bytes (représentatif d'un événement de paiement encodé Avro).
Outils : OpenMessaging Benchmark Framework + Prometheus + Grafana.
Critères : Throughput max soutenable, latence P99/P999, perte de messages sous failover.
Référence : Benchmark reproductible open-source : github.com/openmessaging/benchmark

◆ Résultats Comparatifs

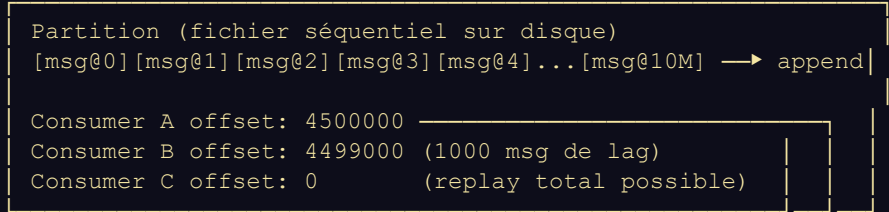
Métrique	Apache Kafka 3.7	RabbitMQ 3.13 (Streams)	Avantage
Throughput max soutenable	14.2M messages/sec	4.1M messages/sec	Kafka ×3.5
Latence P50 (médiane)	1.2ms	0.8ms	RabbitMQ
Latence P99	8ms	45ms (dégradation)	Kafka
Latence P999 (10M TPS)	22ms	>500ms (instable)	Kafka ×22
Perte données (broker crash)	0 (acks=all)	0 (mirroring)	Égalité
Replay de messages	Natif (7 jours)	Non (dead-letter only)	Kafka
Ordering garanti	Par partition	Par queue (FIFO)	Contexte-dépendant
Routage complexe (exchanges)	Limité (partitions)	Natif (topic/fanout/direct)	RabbitMQ
Mémoire par broker sous charge	~8GB (configurable)	~32GB (queue en RAM)	Kafka
Complexité opérationnelle	Élevée (ZooKeeper/KRaft)	Modérée	RabbitMQ

◆ Analyse Architecturale Approfondie

Les chiffres racontent la moitié de l'histoire. L'autre moitié réside dans la conception fondamentale des deux systèmes, qui dicte irrémédiablement leur comportement sous charge extrême :

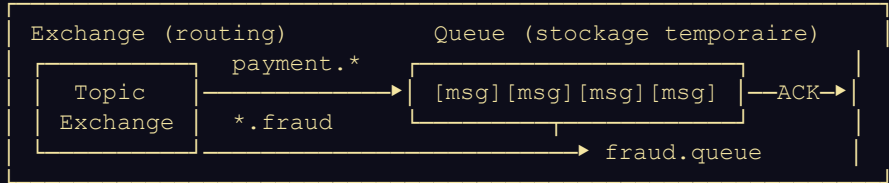
KAFKA vs RABBITMQ : MODÈLE FONDAMENTAL

KAFKA : Log distribué append-only



- Messages jamais détruits après consommation (rétention temps)
- Read = seek sur fichier log = $O(1)$ = vitesse constante

RABBITMQ : Queue transactionnelle avec routing



- Message supprimé après ACK consumer → pas de replay natif
- Routing puissant mais mémoire = bottleneck à 10M TPS

◆ Guide de Décision Architecturale : Kafka ou RabbitMQ ?

Critère de Décision	→ Choisir Kafka	→ Choisir RabbitMQ
Volume	> 100K messages/sec	< 50K messages/sec
Replay nécessaire ?	Oui (audit, reprocessing)	Non (traitement unique)
Ordering strict ?	Par clé (partitions)	Global (queue unique)
Routing complexe ?	Non	Oui (topics, direct, fanout)
Durée de rétention ?	Jours/semaines (log)	Jusqu'à consommation
Latence P50 critique ?	Acceptable ≥ 1 ms	Requis < 1ms
Équipe existante ?	Event streaming, data eng.	Messaging traditionnel
Cas d'usage typique	Event sourcing, CDC, analytics	Task queues, RPC, workflow

◆ Pattern Hybride : Kafka + RabbitMQ en Production

En 2026, l'architecture de référence des scale-ups européens n'est pas un choix binaire — c'est une architecture hybride où chaque outil opère dans sa zone d'excellence. Voici l'implémentation pattern :

```
* JAVA — ARCHITECTURE HYBRIDE KAFKA + RABBITMQ

// — PATTERN HYBRIDE : Kafka pour l'event backbone —
//                               RabbitMQ pour le task routing —

// 1. Kafka : Event sourcing haute performance (paiements, analytics)
@KafkaListener(topics = "payment.raw", groupId = "payment-processor")
public void consumePaymentEvent(PaymentRawEvent event) {
```

```
// Traitement Kafka (high-throughput, ordered by userId)
ProcessedPayment processed = paymentEngine.process(event);

// 2. Routage vers RabbitMQ pour distribution de tâches complexes
// (notifications multi-canal, webhooks clients, retry logic avancé)
NotificationTask task = NotificationTask.builder()
    .userId(event.getUserId())
    .channels(List.of("email", "push", "sms"))
    .priority(processed.isHighValue() ? Priority.HIGH : Priority.NORMAL)
    .build();

// RabbitMQ gère le routage par priorité et canal
rabbitTemplate.convertAndSend(
    "notifications.exchange",
    "notification." + task.getPriority().name().toLowerCase(),
    task
);
}

// — RabbitMQ : Priority Queue pour notifications —
@RabbitListener(queues = "#{highPriorityQueue.name}")
public void processHighPriorityNotification(NotificationTask task) {
    notificationService.sendImmediately(task); // SLA < 2 secondes
}

@RabbitListener(queues = "#{normalPriorityQueue.name}")
public void processNormalNotification(NotificationTask task) {
    notificationService.sendBatched(task); // Batch toutes les 30s
}
```


Synthèse du Chapitre 1 : Les Vérités Architecturales de 2026

Ce chapitre a disséqué les trois piliers de l'architecture moderne avec une profondeur que la plupart des certifications ne couvrent pas en six mois. Voici ce que vous devez porter dans votre ADN d'architecte :

• LES 5 VÉRITÉS ARCHITECTURALES 2026

- 1. Le monolithe ne meurt pas d'un seul coup — il s'effondre progressivement sous le poids de sa propre complexité. La migration Strangler Fig est la seule approche sans risque.
- 2. Kubernetes sans patterns de résilience (probes, affinity, PDB, topology spread) est pire qu'un serveur bare-metal — il offre une fausse sécurité.
- 3. Kafka n'est pas une "grosse RabbitMQ" — c'est un paradigme fondamentalement différent. Confondre les deux est l'erreur d'architecture la plus coûteuse en production.
- 4. À 10M TPS, RabbitMQ sature la mémoire, Kafka lit un fichier séquentiel. La physique ne ment pas.
- 5. L'architecture hybride (Kafka backbone + RabbitMQ task routing) est la réponse pragmatique pour 80% des systèmes de production.

Technologie	Cas d'usage Optimal	Limite à Connaître	Niveau Master
Microservices	Scalabilité indép. par domaine	Complexité opérationnelle ×10	Domain-Driven Design
Kubernetes	Orchestration self-healing	Courbe d'apprentissage 6 mois	Istio + KEDA + KRaft
Kafka	Event sourcing > 100K TPS	Latence P50 > RabbitMQ	Exactly-Once + Streams
RabbitMQ	Task queues, routing complexe	Ne scale pas > 4M TPS	Priority + Dead Letter

Chapitre 2 → Serverless à l'Échelle : AWS Lambda, Cloudflare Workers & Edge Computing